UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Distributed Systems

Indirect communication

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Contents

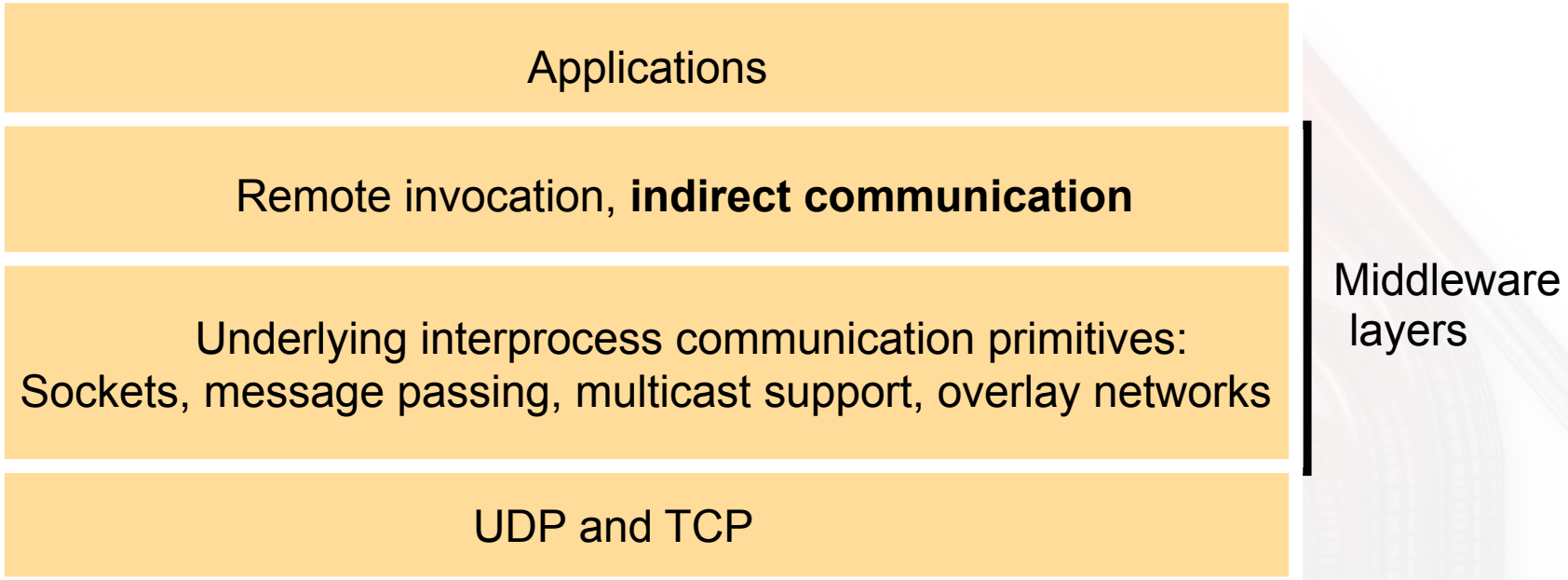
- Indirect vs. direct communication
- Types
- Event-based systems
  - Subscription models
  - IceStorm
- Message queues systems
  - RabbitMQ

Tip

All listings in this topic are available for download at:

`https://github.com/UCLM-ESI/ssdd.examples`

Remote Invocation



© Pearson Education 2012

Indirect vs. direct communication

Direct communication is point-to-point:

- Participants must exist **at same time**.
  - Connection or session establishment
- Each participant require a way to know the address of other.
- Very inefficient when there are many participants.

Request-Reply, RPC/RMI protocols are direct communication.

Indirect vs. direct communication

Indirect communication:

- Communication implies an **intermediary**
- Sender/receiver(s) are decoupled in **time** and **space**:
  - In **space**: sender don't require to know the receiver identities or address, and vice-versa
    - sender/receiver(s) may be transparently replaced, migrated, replicated, etc.
  - In **time**: sender and receiver(s) may have different (even not overlapped) livetimes.
    - sender/receiver(s) don't require to coexist.

Indirect vs. direct communication

	Time-coupled	*Time-uncoupled*
Space coupling	*Properties*: Communication directed towards a given receiver or receivers; receiver(s) must exist at that moment in time *Examples*: Message passing, remote invocation (see Chapters 4 and 5)	*Properties*: Communication directed towards a given receiver or receivers; sender(s) and receiver(s) can have independent lifetimes
Space uncoupling	*Properties*: Sender does not need to know the identity of the receiver(s); receiver(s) must exist at that moment in time *Examples*: IP multicast (see Chapter 4)	*Properties*: Sender does not need to know the identity of the receiver(s); sender(s) and receiver(s) can have independent lifetimes *Examples*: Most indirect communication paradigms covered in this chapter

Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

Suitable for...

- scenarios where users connect and disconnect very often.
  - Mobile environments, messaging services, notification
- scenarios with a large number of users
  - Example: Google Ads system, Spotify
- event dissemination where recipients are unknown and change often
  - Example: RSS, feeds in financial services

Disadvantages

- Overloading due to the introduction of a level of indirection
  - Reliable delivery of messages, order, etc.
- Increased management complexity due to decoupling between transmitters/receivers.
- Difficulty in achieving end-to-end properties
  - Real time, security, etc.

Indirect communication

Communication between the sender and the receiver through an intermediary:

Group communication

- Senders send messages to a "group". They don't know the recipient identities.

Distributed event-based systems

- Disseminate events to multiple recipients through an intermediary (the event broker).

Message queues

- Senders put messages to a distributed queue, and recipients extract them from the queue.

Distributed shared memory

- Simulate centralized shared memory.

Distributed event-based systems
(a.k.a publish-subscribe systems)

Distributed Event-based systems implement one-to-many communications

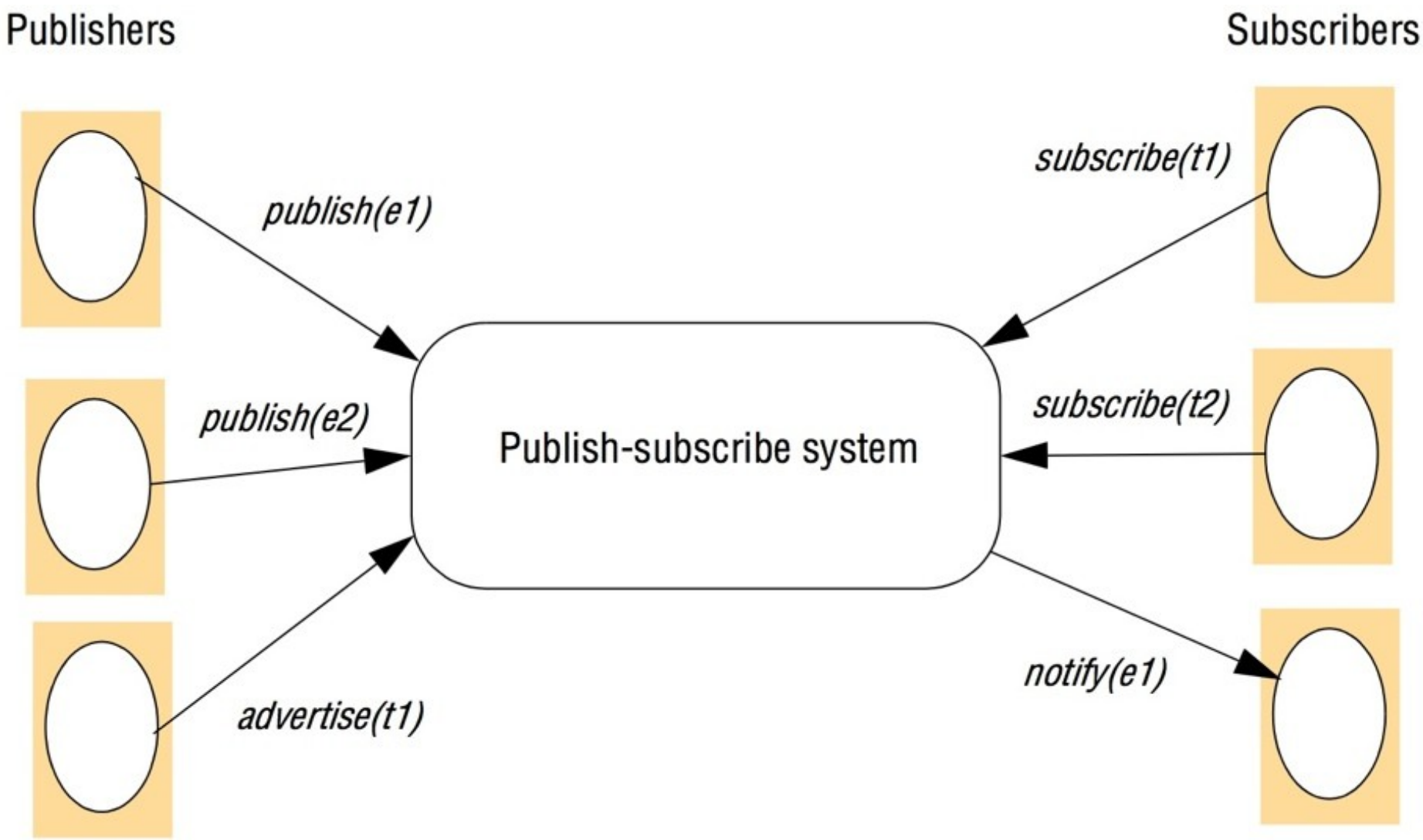
- Heterogeneity & Asynchronism
- Public-subscribe systems
- More decoupled and reactive style than RMI and/or RPC

Examples:
- Financial stock market
- RSS feeds
- GUIs
- Monitoring applications
- Weather, pollution, etc.

Roles:

- Publisher publishes structured events to an event service
- Subscriber expresses interest in particular events through subscriptions to an event service
- Event service or broker receives events from publishers and delivers events to subscribers according to its interests.

Publish-subscribe paradigm



Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

Programming model

- $e \rightarrow$ Event
  - any structured piece of information
- f $\rightarrow$ Filter
  - A pattern to filter the type of events to be received
  - Expressed in specific format (e.g. regular expressions) or language (e.g. SQL-like languages)
- Basic operations:
  - `publish(e)`
  - `subscribe(f)`
  - `unsubscribe(f)`

Subscription models

Channel-based: *publishers* publish events to **named channels** and *subscribers* then subscribe to one of these named channels to receive all events sent to the channel.

- Ex: CORBA Event service, notification service, ZeroC IceStorm

publisher

```
foo = broker.getChannel("foo")
foo.publish(e)
```

PSEUDOCODE

subscriber

```
foo = broker.getChannel("foo")
foo.subscribe(callback)
```

Topic-based (or subject based): Similar to channel-based but a topic (may be hierarchical) is explicitly defined as a **field**.

- Ex: MQTT

publisher

```
event = {
    topic = "temperature",
    value = 30.7}
broker.publish(event)
```

PSEUDOCODE

subscriber

```
broker.subscribe(
    "temperature", callback)
```

Subscription models

Content-based: generalization of topic-based where the filter is a query defined in terms of compositions of constraints over the values of event attributes.

- Ex: DDS

publisher

```
event = {
    color="red", size=12,
    speed=5}
broker.publish(event)
```

PSEUDOCODE

subscriber

```
broker.subscribe(
    color="blue", "size > 8",
    callback)
```

Type-based: subscriptions are defined in terms of types of events and matching is defined in terms of types or subtypes of the given filter.

publisher

```
event = NoiseEvent(120)
broker.publish(event)
```

PSEUDOCODE

subscriber

```
broker.subscribe(
    NoiseEvent, callback)
```

Centralized vs. Distributed

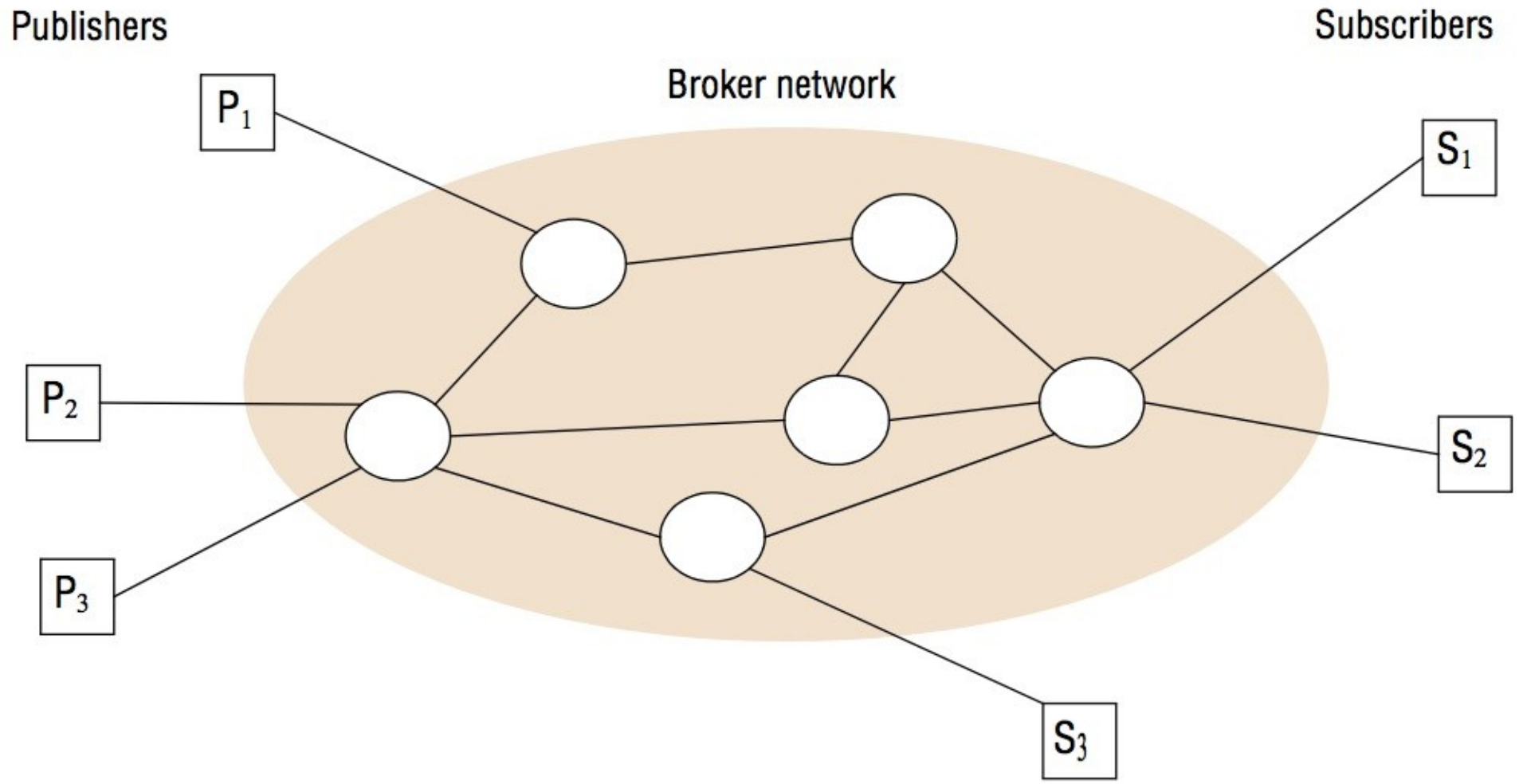
Centralized

- The *broker* is implemented in a single node
- **Pros**: straightforward to implement
- **Cons**: lack of resilience and scalability

Distributed

- A *network of brokers* cooperate for event distribution
- In the extreme case: peer-to-peer
- Channel based and topic based
  - Multicast mechanisms
- Content-based and Type-based can be difficult to implement (need content-based routing)

Broker network



Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

Content based-routing

Event routing performs the task of ensuring that event notifications are routed as efficiently as possible to appropriate subscribers

Content based-routing: exploits content information to efficiently route events to their required destination.

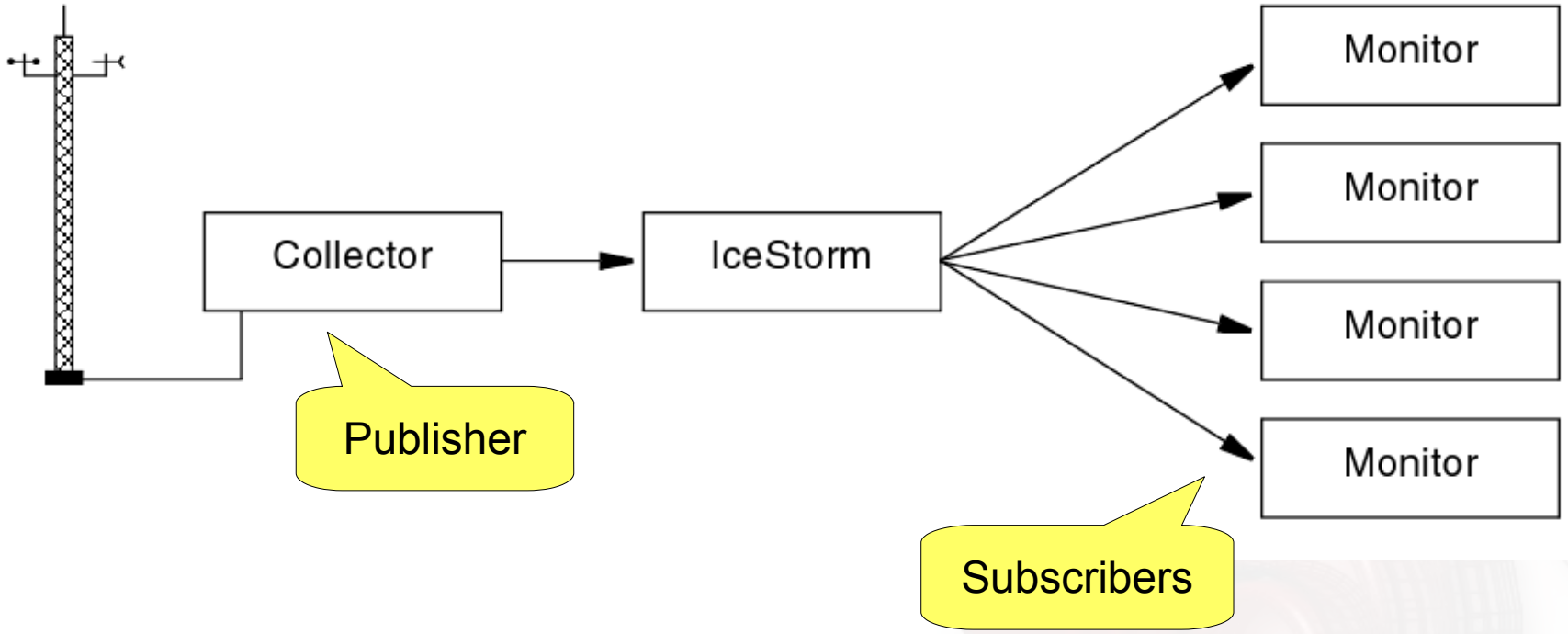
- Flooding
- Filtering
  - Propagation of subscriptions
  - *Match* function
- Advertisements
  - A publisher announces the type of events that generate
- Rendezvous
  - Partition of event space assigning responsibilities of each type of events to specific nodes
  - ~ Filtering and Advertisements

IceStorm
Publish-subscribe example

It is the ZeroC Ice event service

- Invocation oriented (vs. data oriented)
- Topic based
- Federable
- Centralized broker (but replicated: HA-IceStorm)
- Basic filter features

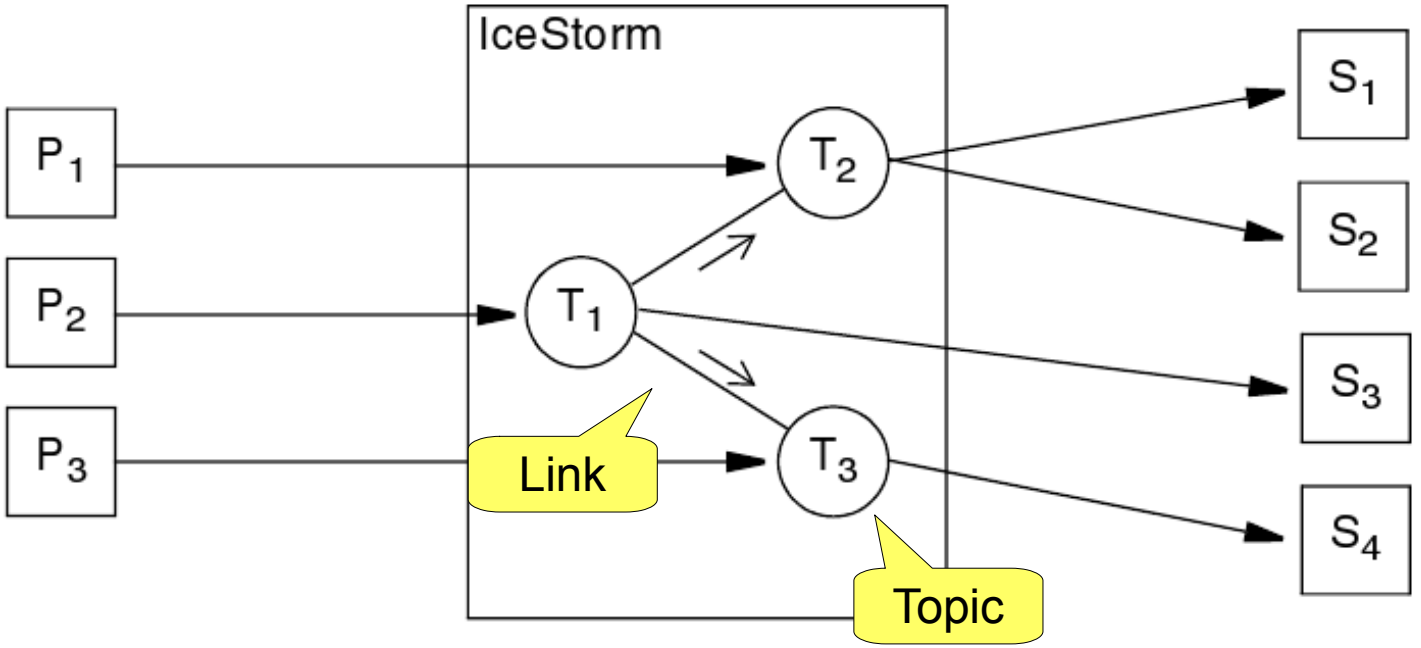
IceStorm



- It publish one time:
  - Broker creates copies
- Publish = invoke oneway method
  - Only provides push

IceStorm
Basic features

- Decouples production and consumption
- Allows to use a different transport protocol for any publisher or subscriber.
- It has a simple filtering mechanism based on weights



IceStorm

DIY

Description: A minimal IceStorm working example

- **[https://github.com/UCLM-ESI/hello.ice/tree/master/py-icestorm]**

Start broker:

```
$ make start
icebox --Ice.Config=icebox.config &
```

See and play with:
- **subscriber.py**
- **publisher.py**

```
$ make run-subscriber
./subscriber.py --Ice.Config=subscriber.config
Using IceStorm in: 'IceStorm.TopicManager.Proxy'
Waiting events... 'CB1EF114-2937-4F8A-BADD-882EAAE203CC ...
Event received: Hello World 0!
```

Run subscriber(s)
(try several)

Run publisher(s)

```
$ make run-publisher
./publisher.py --Ice.Config=publisher.config
Using IceStorm in: 'IceStorm.TopicManager.Proxy'
publishing 10 'Hello World' events
```

Message queues systems

- Distributed message queues decoupled one-to-one communication.
  - A sender put a message into an **explicit** queue.
  - A (single) recipient remove the message from queue.
- Commercial middlewares:
  - WebSphere MQ, Orable AQ
  - JMS (Java Message Service)
  - Amazon SQS
  - Apache Kafka

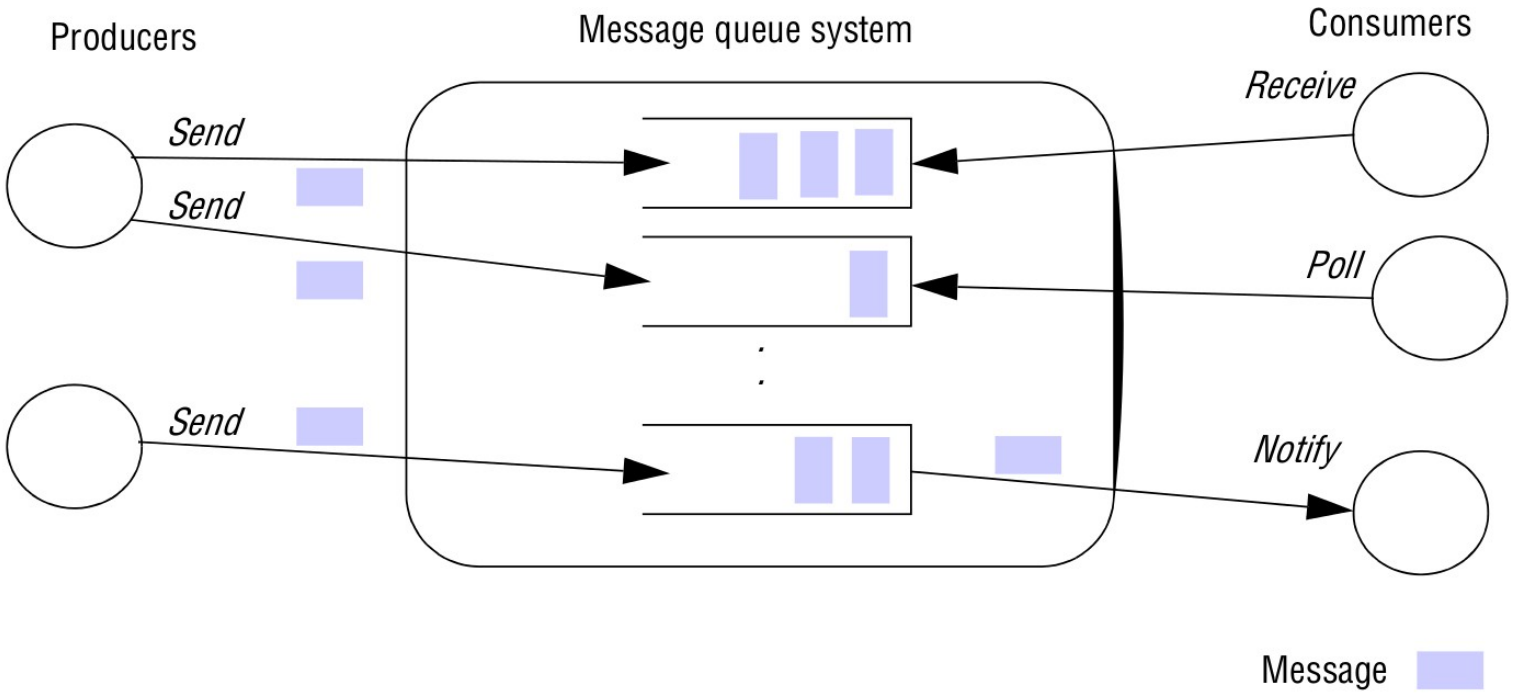
Message queues

- **Queues** are usually FIFO (**First In First Out**), but:
  - Some implementations provide priority or selection criterion.
- Messages consist of:
  - Destination queue identifier
  - Meta data
  - Payload
- Features:
  - Validity (message will be received)
  - Integrity (just one time)
  - Persistent

Programming model

Three styles for receiving messages:

- blocking receive (pull)
- non-blocking receive (polling)
- notify (push)



Working queues



RabbitMQ
Message Queue example

RabbitMQ implements the standard AMQP (**Advanced Message Queuing Protocol**)

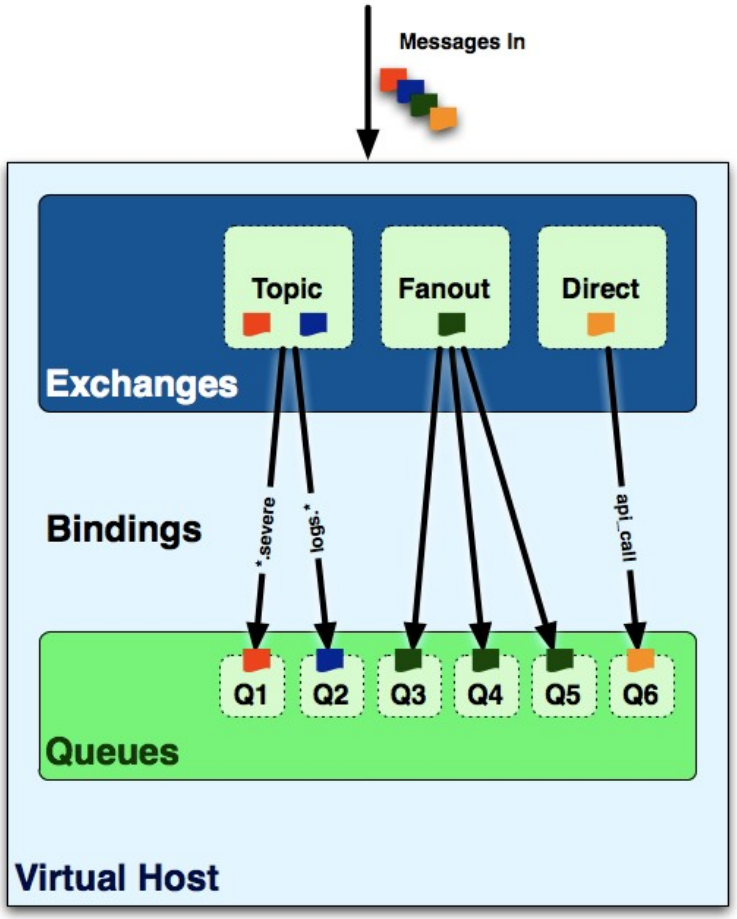
Concepts:

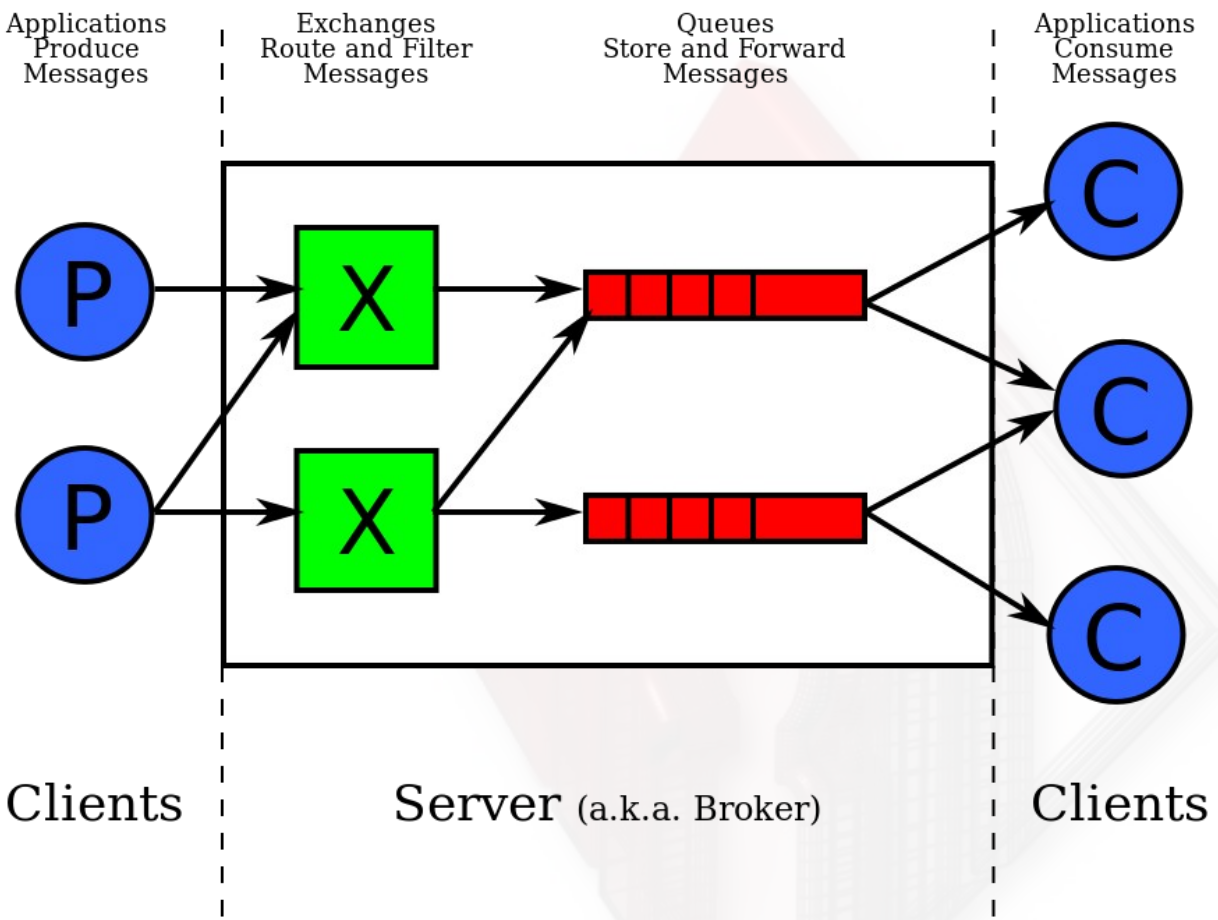
- **Publishers**: send messages to an exchange
- **Exchanges**: implement routing to send messages to consumers.
- **Consumers**: declare queue and bind it to a exchange to receive messages.
- **Queues**: Temporarily store messages from publishers
- **Routing**: Apply routing keys to perform message matching: *direct*, *fan-out* and *topic* exchanges.

It supports several messaging protocols:

- AMQP, STOMP, MQTT and HTTP

RabbitMQ
Message Queue example





http://es.wikipedia.org/wiki/Advanced_Message_Queuing_Protocol
http://blogs.digitar.com/jjww/2009/01/rabbits-and-warrens/

RabbitMQ
producer-consumer

DIY

Description: minimal message queue working example

- **`[ssdd.examples:rabbit-hello]`**



consumer.py

```
import pika

def callback(ch, method, properties, body):
    print " [x] Received %r" % (body,)

localhost = pika.ConnectionParameters(host='localhost')
connection = pika.BlockingConnection()
channel = connection.channel()
channel.queue_declare(queue='hello')
channel.basic_consume(
    queue='hello', no_ack=True, on_message_callback=callback)

print('[*] Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

http://es.wikipedia.org/wiki/Advanced_Message_Queuing_Protocol

RabbitMQ
producer-consumer

DIY

Description: minimal message queue working example

- **[ssdd.examples:rabbit-hello]**



producer.py

```
#!/usr/bin/env python
import pika

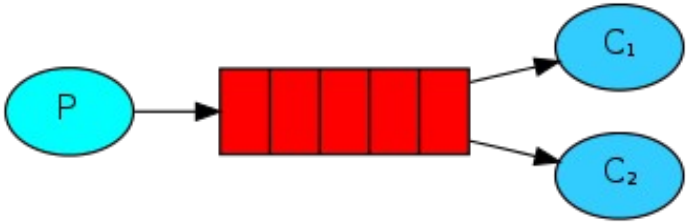
localhost = pika.ConnectionParameters(host='localhost')
connection = pika.BlockingConnection()
channel = connection.channel()

channel.basic_publish(exchange='',
                      routing_key='hello',
                      body='Hello World!')

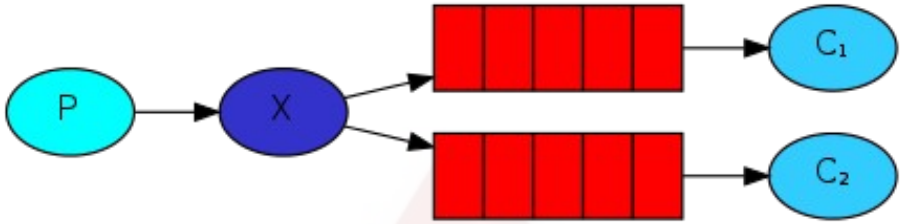
print(" [x] Sent 'Hello World!'")
connection.close()
```

http://es.wikipedia.org/wiki/Advanced_Message_Queuing_Protocol

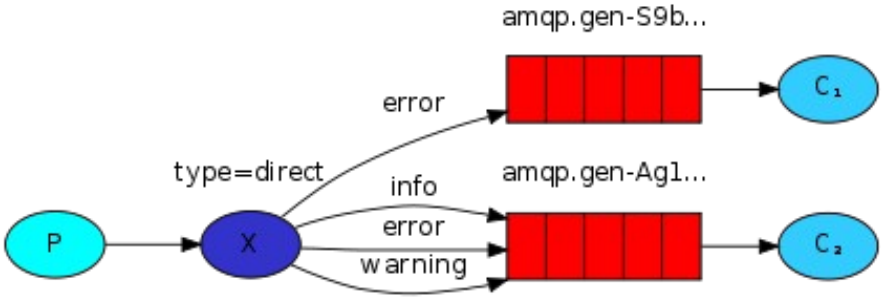
RabbitMQ
flexible binding



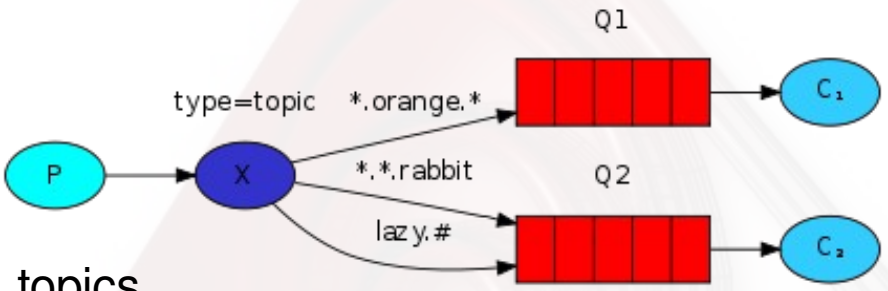
work queues



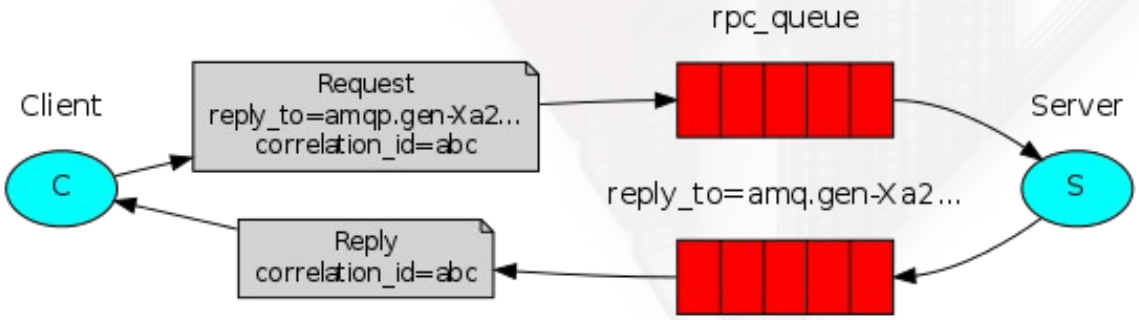
publish-subscribe



routing



topics



RPC

http://www.rabbitmq.com/tutorials/tutorial-one-python.html



Celery
Distributed Task Queues

DIY

Description: minimal task queue working example

- **`[ssdd.examples:celery]`**

tasks.py

```
from celery import Celery
app = Celery('tasks', broker='amqp://')

@app.task
def add(x, y):
    return x + y
```

```
$ celery -A tasks worker
```

client.py

```
from tasks import add

result = add.delay(4, 4)
value = result.get(timeout=1)
```

```
$ ./client.py
```

http://celery.readthedocs.org/en/latest/getting-started/first-steps-with-celery.html

Apache Kafka
Event streaming example

Apache Kafka is a replicated event streaming server.

Concepts:

- **Event**: captured fact/data to be stored in a log.
- **Event streaming**: practice of capturing data in real time from event sources and store them for later retrieval.
- **Producer**: entity that send events to a log.
- **Consumer**: entity that read events from the log.
- **Topic**: A category of events.
- **Partition**: subset of a topic, mainly used to allow parallelization.
- **Broker:** store and manage events (it uses a cluster of brokers).

Apache Kafka
Event streaming example

Events contains at least:

- Key
- Value
- Timestamp

When an event is stored in a topic, an incremental id (**offset**) is assigned for the partition.

Consumers uses the offset to “remember” the last item that was read from a partition.

To allow this recovery mechanism and parallelism, Kafka uses **consumer groups**. When a consumer joins a consumer group, it will be assigned to read exclusively a subset of partitions.

In motion: https://softwaremill.com/kafka-visualisation/

Apache Kafka
Event streaming example

For event consuming there are 2 different approaches:

- **Lambda**: Using the topic as storage: Consumer run sporadically. It will consume a number of events, then stops.
- **Kappa**: Consume at the same time the data is produced (standard).

References

G. Coulouris, *Distributed Systems: Concepts and Design*, Addison Wesley 2011, Fifth edition

- Section 6.1 – Introduction
- Section 6.3 – Publish-subscribe systems
- Section 6.4 – Message queues